

MODULE 05 · PART 2 - WINDOW FUNCTIONS

The Frame Clause

The third question, the only default that changes an answer, and the reason one engine refuses to have one.

PART 2 of 8 MODULE 5 of 26 RUNS about 40 min

Why this one is the longest

- ◆ **It is the only default that changes a number.** The other two change shape, which you notice immediately.
- ◆ **It produces answers that reconcile.** Totals agree, boundaries agree, and the middle of the report is wrong.
- ◆ **And the engines disagree about it,** so the same query means different things in two systems you own.
- ◆ **One line of SQL removes all of it.** The length of this module is about earning the habit, not about complexity.

THE TAKEAWAY

Nothing here is difficult. It is long because the failure is quiet, plausible and portable, and those three together defeat ordinary review.

The partition is what a row **could** see. The frame is what it **does** see.

Recomputed for every row

```
1  -- row 4 of the partition, frame: ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
2
3  row 1    120      out of frame
4  row 2    340      in frame  <-- start
5  row 3    180      in frame
6  row 4     90      in frame  <-- current row, end
7  row 5   410      out of frame
8
9  -- SUM over the frame = 610
```

6 Move to row 5 and the whole frame slides down one. Rows leave at the top as they enter at the bottom.

THE TAKEAWAY

A frame is not computed once for the partition. It is redrawn around every row, which is what makes a running total run.

What a frame is

- ◆ **A start and an end**, both expressed relative to the current row, recomputed for every row.
- ◆ **It only exists when the function is frame-sensitive**. Aggregates, FIRST_VALUE, LAST_VALUE, NTH_VALUE.
- ◆ **And only when the partition has an ordering**. Without one there is no 'before' and 'after' to measure against.
- ◆ **Three units to measure in: ROWS, RANGE and GROUPS**. They are not interchangeable and they are one word apart.

THE TAKEAWAY

A frame is recomputed for every row. That is what makes a running total run, and it is also what makes a sliding frame expensive.

Reading a frame out loud

WRITTEN

SAID

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

everything up to and including me

ROWS BETWEEN 6 PRECEDING AND CURRENT ROW

the six before me, and me

RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

everything up to and including everyone equal to me

ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING

me, and everything after me

RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW

the last seven days, whatever that is

THE TAKEAWAY

Row three is the sentence people never say. Read a default frame out loud in those words and you would rarely have written it.

One function, three frames, three answers

```
1 SELECT sold_ts, net_amount,  
2     SUM(net_amount) OVER (ORDER BY sold_ts  
3         ROWS UNBOUNDED PRECEDING) AS running,  
4     SUM(net_amount) OVER (ORDER BY sold_ts  
5         ROWS BETWEEN 2 PRECEDING  
6         AND CURRENT ROW) AS last_three,  
7     SUM(net_amount) OVER (ORDER BY sold_ts  
8         ROWS BETWEEN UNBOUNDED PRECEDING  
9         AND UNBOUNDED FOLLOWING) AS whole_partition  
10 FROM fact_ledger
```

3 Same function, same ordering, same partition. Only the frame differs, and every column is a different measure.

THE TAKEAWAY

A running total, a three-row moving sum, and a grand total. Nothing distinguishes them except the frame, and two of the three are one line long.

The anatomy of a frame clause

PART	WHAT IT CAN BE
<code>unit</code>	ROWS, RANGE or GROUPS. What the offsets are counted in.
<code>start</code>	UNBOUNDED PRECEDING, n PRECEDING, CURRENT ROW, n FOLLOWING.
<code>end</code>	n PRECEDING, CURRENT ROW, n FOLLOWING, UNBOUNDED FOLLOWING.
<code>BETWEEN</code>	Required to state an end. Omit it and the end is CURRENT ROW.
<code>EXCLUDE</code>	Optional, and rarely supported. Removes rows from the frame.

THE TAKEAWAY

Row four is the one that catches people. ROWS 5 PRECEDING does not mean five rows. It means five rows before, through to here, which is six.

Always write BETWEEN

- ◆ **A frame with no BETWEEN silently ends at the current row.** That is in the standard, and every engine does it.
- ◆ **So ROWS 6 PRECEDING is a seven-row window,** not a six-row one, and it is never written down as seven anywhere.
- ◆ **Writing BETWEEN makes both ends visible,** which means a reviewer can check the arithmetic without knowing the rule.
- ◆ **It costs six characters.** There is no performance difference and no behavioural difference, only a readable one.

THE TAKEAWAY

Off-by-one in a moving average is the most common numerical bug in analytics SQL, and the shorthand is what hides it.

ROWS counts rows. A physical offset, from the current row, regardless of **what those rows contain.**

No ordering, no frame

- ◆ **An aggregate window with no ORDER BY sees the whole partition.** Every row gets the same value.
- ◆ **That is a frame too**, formally unbounded in both directions, and it is the one default that surprises nobody.
- ◆ **Writing a frame without an ORDER BY is an error.** There is no sequence to measure an offset against.
- ◆ **So the order matters twice over.** Adding ORDER BY both sequences the rows and creates a frame where there was none.

THE TAKEAWAY

SUM OVER (PARTITION BY x) and SUM OVER (PARTITION BY x ORDER BY y) are not the same measure. One is a total and the other is a running total.

The same function, with and without an order

```
1 SELECT store_key, sale_date, daily_sales,  
2        SUM(daily_sales) OVER (  
3            PARTITION BY store_key  
4        ) AS store_total,  
5        SUM(daily_sales) OVER (  
6            PARTITION BY store_key  
7            ORDER BY sale_date  
8            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
9        ) AS store_running  
10 FROM agg_store_day;
```

7 One added line changes a total into a running total. The frame on line 8 is what makes that explicit.

THE TAKEAWAY

Without line 8 this would still be a running total on most engines, and nothing on the page would say so. That is the whole argument for writing it.

The running total, written properly

```
1 SELECT posted_ts, amount,  
2       SUM(amount) OVER (  
3         PARTITION BY account_key  
4         ORDER BY posted_ts, ledger_key  
5         ROWS BETWEEN UNBOUNDED PRECEDING  
6                   AND CURRENT ROW  
7       ) AS running_balance  
8 FROM fact_ledger;
```

- 4 A total order, from module two. Without the ledger key, tied timestamps make the balance non-deterministic.
- 5 An explicit ROWS frame. One row at a time, whatever the timestamps do.

THE TAKEAWAY

Two lines of care. Together they turn a running balance from something that usually agrees with the ledger into something that always does.

The ROWS vocabulary

FRAME

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

ROWS BETWEEN 6 PRECEDING AND CURRENT ROW

ROWS BETWEEN 3 PRECEDING AND 3 FOLLOWING

ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING

ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING

ROWS BETWEEN 1 FOLLOWING AND 3 FOLLOWING

WHAT EACH ROW SEES

Everything up to here. A running total.

This row and the six before it. A trailing window.

Seven rows, centred. A smoothed series.

Everything from here on. A remaining total.

The whole partition, on every row.

The next three rows, excluding this one.

THE TAKEAWAY

Six frames, six different measures, one function. Learning these six is most of what there is to learn about frames.

Four shapes of accumulation

- ◆ **Running total.** UNBOUNDED PRECEDING to CURRENT ROW. Climbs to the partition total on the last row.
- ◆ **Remaining total.** CURRENT ROW to UNBOUNDED FOLLOWING. Starts at the partition total and falls to the last value.
- ◆ **Trailing window.** n PRECEDING to CURRENT ROW. Flat-ish, and it forgets.
- ◆ **Centred window.** n PRECEDING to n FOLLOWING. Smooth, and it uses information from the future.

THE TAKEAWAY

The fourth is fine for analysis and wrong for anything operational. A centred average on today's row needs rows that have not happened yet.

A seven-day trailing average

```
1 SELECT date_key, daily_sales,  
2        AVG(daily_sales) OVER (  
3            PARTITION BY store_key  
4            ORDER BY date_key  
5            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  
6        ) AS avg_7d,  
7        COUNT(*) OVER (  
8            PARTITION BY store_key  
9            ORDER BY date_key  
10           ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  
11        ) AS days_in_avg  
12 FROM agg_store_day;
```

10 The same frame, counted. The first six rows of each store return 1 through 6, not 7.

THE TAKEAWAY

The early rows are averages of fewer days, not nulls. Publishing them unlabelled means a store's first week always looks unusually smooth.

Running, remaining, and percent complete

```
1 SELECT sale_date, daily_sales,  
2        SUM(daily_sales) OVER (ORDER BY sale_date  
3        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS so_far,  
4        SUM(daily_sales) OVER (ORDER BY sale_date  
5        ROWS BETWEEN 1 FOLLOWING AND UNBOUNDED FOLLOWING) AS to_come,  
6        SUM(daily_sales) OVER (ORDER BY sale_date  
7        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)  
8        / SUM(daily_sales) OVER () AS pct_complete  
9 FROM agg_store_day WHERE store_key = 41;
```

5 One FOLLOWING, so the current row is excluded and so_far plus to_come equals the total exactly.

THE TAKEAWAY

Three frames over one sort. The percentage divides a framed window by an unframed one, which is the shape behind every percent-of-total measure.

Centred frames and their edges

- ◆ **A centred window is shorter at both ends of the partition.** The first row has no preceding rows and the last has no following ones.
- ◆ **Which makes the series start and end unusually smooth,** because those points are averaged over fewer observations.
- ◆ **It also cannot be used for a forecast,** since the current row's value depends on rows that do not exist yet.
- ◆ **Trailing frames have only one bad edge,** and that is one reason operational reporting nearly always uses them.

THE TAKEAWAY

Centred for analysis of the past, trailing for anything that runs today. The frame encodes that decision, and it is the only place it is recorded.

A centred seven-day average

```
1 SELECT sale_date, daily_sales,  
2        AVG(daily_sales) OVER (  
3            PARTITION BY store_key  
4            ORDER BY sale_date  
5            ROWS BETWEEN 3 PRECEDING AND 3 FOLLOWING  
6        ) AS smoothed,  
7        COUNT(*) OVER (  
8            PARTITION BY store_key ORDER BY sale_date  
9            ROWS BETWEEN 3 PRECEDING AND 3 FOLLOWING  
10       ) AS n_days  
11 FROM agg_store_day;
```

5 Seven rows, centred. The first three and last three rows of each store return counts of 4, 5 and 6.

THE TAKEAWAY

Carrying the count is the habit worth forming. It turns an invisible edge effect into a column you can filter on.

RANGE counts **values**. Every row with the same ORDER BY value is in the frame together.

One word, two meanings

ROWS

Positional

```
1 SUM(amount) OVER (  
2   ORDER BY sold_date  
3   ROWS BETWEEN  
4   UNBOUNDED PRECEDING  
5   AND CURRENT ROW  
6 )
```

CURRENT ROW means this row. Tied rows accumulate one at a time, in whatever order they were sorted.

RANGE

Value-based

```
1 SUM(amount) OVER (  
2   ORDER BY sold_date  
3   RANGE BETWEEN  
4   UNBOUNDED PRECEDING  
5   AND CURRENT ROW  
6 )
```

CURRENT ROW means this row and every row sharing its date. All of them read the same total.

THE TIEBREAKER

Identical queries apart from one word. On data where the ordering column is unique, they return identical answers, which is exactly why nobody notices.

Kestrel's tied timestamps

```
1 SELECT sold_ts, net_amount,  
2       SUM(net_amount) OVER (  
3         PARTITION BY txn_id  
4         ORDER BY sold_ts  
5         RANGE BETWEEN UNBOUNDED PRECEDING  
6           AND CURRENT ROW  
7       ) AS running_line_total  
8 FROM fact_sales_line  
9 WHERE store_key = 41 AND date_key = 20260831;
```

- 4 sold_ts has second precision. 31% of Kestrel's transactions have two or more lines inside one second.
- 5 Those lines are peers, so each one reads the total of all of them. The running total jumps and then stalls.

THE TAKEAWAY

Roughly a third of transactions produce a running total that is right at the end and wrong in the middle. Every receipt still totals correctly.

The same four lines, two frames

LINE AND TIME	ROWS RUNNING, RANGE RUNNING
1 12:04:07 \$3.20	3.20 3.20
2 12:04:08 \$1.80	5.00 9.10
3 12:04:08 \$2.50	7.50 9.10
4 12:04:08 \$1.60	9.10 9.10
5 12:04:09 \$4.40	13.50 13.50
6 12:04:10 \$2.00	15.50 15.50

THE TAKEAWAY

Rows 2, 3 and 4 share a second. Under RANGE all three read 9.10, which is the total after row 4. The first row and the last row agree in both columns.

On most engines, **RANGE is the default**. You get it by writing an ORDER BY and nothing else.

Why it is invisible

- ◆ **With a unique ordering, RANGE and ROWS give identical results.** So the query is correct when it is written and reviewed.
- ◆ **The defect is latent.** It activates when the data develops ties, which can be years later and is nobody's deployment.
- ◆ **And it is partial.** The last row of each tied group is right, so totals reconcile and only the middle is wrong.
- ◆ **Writing ROWS explicitly removes the whole category,** forever, at a cost of one line per window.

THE TAKEAWAY

A bug that is correct on the test data, correct at the boundaries, and wrong in between. It is close to the worst shape a defect can have.

Why the standard chose RANGE

- ◆ **Because a set has no order, and peers are genuinely indistinguishable.** RANGE is the answer that does not depend on a sort.
- ◆ **ROWS gives a different answer depending on how ties were arranged,** which is a property the standard was trying to avoid.
- ◆ **So RANGE is the principled default and ROWS is the useful one.** Both statements are true at once.
- ◆ **The mistake is not the choice.** It is that the choice is invisible, and almost nobody knows it was made.

THE TAKEAWAY

RANGE is defensible. If two rows are equal on everything the query can see, returning different answers for them is arguably the stranger behaviour.

Redshift has **no RANGE at all**. And it refuses to guess a frame for you.

Frame support, by engine

ENGINE	ROWS, RANGE, GROUPS, EXCLUDE
PostgreSQL 11+	All four. The most complete implementation in common use.
Oracle	ROWS and RANGE with offsets. No GROUPS, no EXCLUDE.
Snowflake, BigQuery	ROWS and RANGE. No GROUPS, no EXCLUDE.
SQL Server	ROWS and RANGE, but RANGE takes no offsets. No GROUPS.
Amazon Redshift	ROWS only. No RANGE, no GROUPS, no EXCLUDE.
DuckDB, SQLite	All four, following PostgreSQL closely.

THE TAKEAWAY

Redshift is the odd one out and it is the safe one. Everywhere else, an ORDER BY with no frame gives you RANGE without asking.

A default, or an error

MOST ENGINES

Supply a frame silently

- ORDER BY alone is legal
- You get RANGE, unmasked
- Correct until something ties
- No warning, ever

REDSHIFT

Refuse to guess

- ORDER BY alone is an error
- You must write the frame
- The only unit is the safe one
- The failure is at compile time

THE TIEBREAKER

A missing feature that functions as a safety rail. Redshift cannot produce this defect, because it will not let you write the query that has it.

Which means the direction of a port matters

- ◆ **Postgres to Redshift is safe.** A query relying on the default simply fails, loudly, before it runs.
- ◆ **Redshift to Postgres is not.** The explicit ROWS frame looks like clutter, and removing it is a silent change of meaning.
- ◆ **Nobody reviews a deletion of six words** that the target engine does not require. It reads as tidying up.
- ◆ **Kestrel ports in exactly that direction**, from the Redshift warehouse to the Postgres store-ops replica.

THE TAKEAWAY

One direction turns a latent defect into a compile error. The other turns an explicit clause into a latent defect. Same two engines, same two queries.

Spotting it in review

- ◆ **Search for OVER followed by ORDER BY with no ROWS, RANGE or GROUPS.** Every hit is a default you did not choose.
- ◆ **Then ask one question of each: is the ordering column unique?** If yes it is currently fine. If no, it is currently wrong.
- ◆ **Currently fine is not fine.** Uniqueness is a property of today's data, and nothing enforces it tomorrow.
- ◆ **Ranking functions are false positives.** They take no frame, so an ORDER BY with no frame is correct there.

THE TAKEAWAY

This search takes a minute across a whole repository and it is not a standard review item anywhere. It should be.

What to look for

```
1 -- flag this
2 SUM(amount) OVER (PARTITION BY a ORDER BY b)
3 AVG(x)        OVER (ORDER BY ts)
4 LAST_VALUE(p) OVER (PARTITION BY k ORDER BY d)
5
6 -- ignore this
7 ROW_NUMBER() OVER (PARTITION BY a ORDER BY b, id)
8 RANK()        OVER (ORDER BY units DESC)
9 SUM(amount)   OVER (PARTITION BY a)
```

- 4 The third one is scenario two. An ORDER BY with no frame, on a function whose whole purpose needs the frame to reach the end.

THE TAKEAWAY

Three to flag, three to ignore, and the difference is whether the function takes a frame and whether an ordering was supplied.

A running ledger balance was written against Kestrel's Redshift warehouse with an explicit ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW frame. It is being ported to the Postgres store-ops replica for a new operational dashboard.

During the port an engineer removes the frame clause, noting that Postgres does not require it and the query returns the same numbers in testing. Two months later, finance reports that intraday balances plateau at several points each day, though the closing balance is always right.

What happened?

- ☐ A The port dropped the tiebreaker from the ORDER BY, so rows are accumulated in a different sequence.
- ☐ B Postgres supplied a RANGE default once the explicit frame was removed, so tied postings share one cumulative total.
- ☐ C Postgres and Redshift sort nulls at opposite ends, which changes the order in which postings accumulate.
- ☐ D A running balance is not expressible as a window function and needs a recursive CTE to be correct.

What happened?

A The tiebreaker was dropped

Would make the balance non-deterministic between runs rather than plateau within a run. Real, and it produces a different symptom.

C Nulls sort differently

Both engines put nulls at the same end, being Postgres-derived. And a plateau is not what a reordering looks like.

B Postgres supplied RANGE

Tied postings became peers, so each reads the total of all of them. The last one is right, which is why the close reconciles.

D It needs recursion

It does not. The Redshift version was a window function and was correct. Recursion is for feedback, which a running total has none of.

THE SIGNAL

"The closing balance is always right." A defect that is wrong in the middle and correct at the end is the signature of peers under a RANGE frame, and of almost nothing else.

LAST_VALUE returns **the current row**. It is not broken, and it is the most-asked question in this subject.

The query everybody writes

```
1 SELECT product_key, effective_from, price,  
2        FIRST_VALUE(price) OVER (  
3          PARTITION BY product_key ORDER BY effective_from  
4        ) AS first_price,  
5        LAST_VALUE(price) OVER (  
6          PARTITION BY product_key ORDER BY effective_from  
7        ) AS last_price  
8 FROM fact_price_change;
```

- 4 FIRST_VALUE works. The frame starts at UNBOUNDED PRECEDING, so the first row is always in it.
- 7 LAST_VALUE does not. The frame ends at CURRENT ROW, so the last row in it is the current one.

THE TAKEAWAY

Both functions are behaving identically and correctly. One of them happens to be asking for a row the default frame includes, and the other is not.

Two ways to fix it

STATE THE FRAME

Say what you meant

```
1 LAST_VALUE(price) OVER (  
2   PARTITION BY product_key  
3   ORDER BY effective_from  
4   ROWS BETWEEN UNBOUNDED PRECEDING  
5               AND UNBOUNDED FOLLOWING  
6 )
```

Explicit, portable, and it says on the page that the whole partition is in scope.

TURN IT AROUND

Use FIRST_VALUE backwards

```
1 FIRST_VALUE(price) OVER (  
2   PARTITION BY product_key  
3   ORDER BY effective_from DESC  
4 )
```

Shorter, and it works by relying on the same default that caused the problem.

THE TIEBREAKER

Both return the right answer. Only one of them leaves the next reader able to see why, and the other needs a second sort if the query also ranks ascending.

Why FIRST_VALUE never complains

- ◆ **The default frame starts at UNBOUNDED PRECEDING.** The first row of the partition is inside it, always.
- ◆ **So FIRST_VALUE is right for the wrong reason,** and it teaches people that the default is harmless.
- ◆ **Which is why LAST_VALUE feels like a bug.** Two functions written identically, one working and one not.
- ◆ **Write the frame on both.** Then neither depends on a default, and the pair reads consistently.

THE TAKEAWAY

FIRST_VALUE working by accident is the reason LAST_VALUE is so confusing. The asymmetry is in the frame, not in the functions.

Three more things about LAST_VALUE

- ◆ **With ties, the fixed version returns the last peer**, chosen arbitrarily unless the ordering is total. Module two, again.
- ◆ **On SQL Server it costs an on-disk spool**, which is why the same query is markedly slower there than the FIRST_VALUE form.
- ◆ **MAX is often what was actually wanted**. Latest price and highest price differ, and people frequently mean the latter.
- ◆ **IGNORE NULLS changes it again**, skipping nulls to find the last non-null value. That is module six.

THE TAKEAWAY

Once the frame is stated, LAST_VALUE is ordinary. Every remaining surprise in it is a determinism question, which you already know how to answer.

A pricing report shows each product's price history with a column headed 'price at period end', computed with `LAST_VALUE` over `effective_from` within each product. It runs on the Postgres replica.

A reviewer notices that the period-end column is identical to the price column on every single row of the report, for all 84,000 products. The query returns no errors and the row count is correct.

What is the cause?

- ☐ A `LAST_VALUE` requires `IGNORE NULLS` to skip the rows where no price was recorded.
- ☐ B The ordering is ascending. It should be descending, with `FIRST_VALUE` used instead.
- ☐ C The partition is wrong, so each row is alone in its own partition and is therefore its own last value.
- ☐ D The default frame ends at the current row, so the last value within the frame is the current row.

◆ PAUSE AND DECIDE

What is the cause?

A It needs IGNORE NULLS

Changes which rows are candidates and does nothing about where the frame ends. The current row would still be the last one in it.

C The partition is wrong

Would show as one row per partition everywhere. The report has a full price history per product, so the partitions are plainly correct.

B Reverse it and use FIRST_VALUE

This does produce the right answer, which is what makes it dangerous. It fixes the symptom without naming the cause, so every other window in the query keeps it.

D The frame ends at the current row

RANGE UNBOUNDED PRECEDING AND CURRENT ROW is supplied by default. The last row in that frame is always the current one.

THE SIGNAL

"Identical on every single row." Not wrong sometimes, or wrong for some products. A column that exactly equals another column is a frame problem, not a data problem.

RANGE with an offset is a **value** window.
Ninety days means ninety days, not ninety rows.

Peers, or positions

YOU WANT PEERS

RANGE

- Equal rows must read the same
- The value is the identity
- Offsets need one numeric column
- Gaps in the series are handled

YOU WANT POSITIONS

ROWS

- Each row counted separately
- Position is the identity
- Offsets work on any ordering
- Gaps are invisible

THE TIEBREAKER

Ask whether two rows with the same ordering value should get the same answer. That one question picks the unit, and it is a question about the report rather than about SQL.

When peers are exactly what you want

- ◆ **Ranking-style cumulative measures.** Share of products at or below this price should not depend on tie order.
- ◆ **Daily totals from timestamped rows.** Every row of one day reading that day's total is often the point.
- ◆ **But say so explicitly.** Write RANGE, and the reader knows you meant peers rather than inherited them.
- ◆ **CUME_DIST is this idea as a function,** and module nine covers it. It is RANGE thinking with no frame to write.

THE TAKEAWAY

RANGE is right more often than its reputation in this module suggests. What is never right is getting it without knowing.

Ninety days, not ninety rows

```
1 SELECT store_key, sale_date, daily_sales,  
2        AVG(daily_sales) OVER (  
3            PARTITION BY store_key  
4            ORDER BY sale_date  
5            RANGE BETWEEN INTERVAL '89 days' PRECEDING  
6                        AND CURRENT ROW  
7        ) AS avg_90d  
8 FROM agg_store_day;
```

- 5 An interval offset. The frame is defined by dates, so missing days simply contribute nothing.
- 6 Postgres 11 and Oracle do this properly. Redshift cannot express it, and SQL Server takes no RANGE offsets.

THE TAKEAWAY

This is the frame ROWS cannot write. If a store closed for nineteen days, ROWS BETWEEN 89 PRECEDING reaches back a hundred and eight calendar days instead.

What a RANGE offset requires

- ◆ **Exactly one ORDER BY column**, because the offset is arithmetic on that value and there is nothing to add to a pair.
- ◆ **A type that supports addition**. A number, a date, a timestamp. Not a string, not a composite key.
- ◆ **Which means no tiebreaker column**. The one place in this course where you cannot make the ordering total.
- ◆ **That is acceptable here**, because RANGE includes all peers by design. Ties do not need breaking, they are the point.

THE TAKEAWAY

The one exception to the rule from module two. Under a RANGE offset frame, an incomplete ordering is not a defect, because peers are deliberately treated as one.

Frame shapes and what they are called

FRAME	THE MEASURE IT PRODUCES
UNBOUNDED PRECEDING AND CURRENT ROW	Running total, cumulative sum, balance
n PRECEDING AND CURRENT ROW	Moving average, trailing sum, rolling window
n PRECEDING AND n FOLLOWING	Smoothed series, centred average
CURRENT ROW AND UNBOUNDED FOLLOWING	Remaining total, backlog, run-off
UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING	Partition total, share denominator
1 FOLLOWING AND UNBOUNDED FOLLOWING	Everything still to come, excluding today

THE TAKEAWAY

Business people name the measure and engineers write the frame. This table is the translation, and most disagreements live in row two.

A store performance dashboard shows a seven-day trailing average of daily sales, computed with `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` over a table holding one row per store per trading day.

After a store reopens from a nineteen-day refurbishment, its trailing average for the following week is visibly wrong, blending pre-closure and post-reopening trade. Sixty-three stores have closed for refurbishment since 2022.

What is the correct fix?

- A** `ROWS` counts rows, not days. Use a `RANGE` interval frame, or complete the series against `dim_date` before averaging.
- B** Widen the frame to `ROWS BETWEEN 20 PRECEDING AND CURRENT ROW`, so a typical closure is fully covered.
- C** Exclude the closure period from the source table, so the average is computed over trading days only.
- D** Use `GROUPS BETWEEN 6 PRECEDING AND CURRENT ROW`, which counts distinct dates rather than rows.

◆ PAUSE AND DECIDE

What is the correct fix?

- A

 RANGE interval, or a date spine
The frame has to be defined in the unit the measure is named in. Seven days means seven days, and only one of the three units can say that.
- B

 Widen the frame
Changes a seven-day average into a twenty-one-day one for every store that never closed. It fixes 63 stores by breaking 412.
- C

 Exclude the closure
The closure is already absent. There are no rows for those days, which is exactly why ROWS reaches past them.
- D

 Use GROUPS
Counts distinct values that are present, so it still skips the closed days. It also does not exist on Redshift, where this table lives.

THE SIGNAL

"One row per store per trading day." A table with no row for a closed day cannot be counted in days by a frame that counts rows. The gap is invisible to ROWS.

RANGE on a number, not only a date

- ◆ The offset is arithmetic, so any numeric column works. Price, quantity, score, distance.
- ◆ **RANGE BETWEEN 5 PRECEDING AND 5 FOLLOWING on price** gives every product the average of products within five dollars of it.
- ◆ That is a neighbourhood, not a window, and it cannot be expressed with ROWS at any width.
- ◆ It is the natural tool for banding, where the question is about similar values rather than adjacent rows.

THE TAKEAWAY

RANGE is not a date feature. It is a value feature, and the date case is simply the one everybody meets first.

A price neighbourhood

```
1 SELECT product_key, price,  
2        AVG(units_sold) OVER (  
3            PARTITION BY category_key  
4            ORDER BY price  
5            RANGE BETWEEN 5 PRECEDING AND 5 FOLLOWING  
6        ) AS avg_units_nearby,  
7        COUNT(*) OVER (  
8            PARTITION BY category_key ORDER BY price  
9            RANGE BETWEEN 5 PRECEDING AND 5 FOLLOWING  
10       ) AS products_nearby  
11 FROM vw_product_price_units;
```

- 5 Products priced within five dollars either way, however many that is. In a dense band it might be 200, and in a sparse one, one.

THE TAKEAWAY

The count is doing real work here. An average over one neighbour is not comparable to an average over two hundred, and the frame cannot tell you which you got.

GROUPS counts **distinct value groups**. It is the compromise between the other two, and almost nobody has it.

Three distinct days back

```
1 SELECT sale_date, store_key, daily_sales,  
2        SUM(daily_sales) OVER (  
3            ORDER BY sale_date  
4            GROUPS BETWEEN 3 PRECEDING  
5                        AND CURRENT ROW  
6        ) AS last_four_distinct_days  
7 FROM agg_store_day;
```

4 Three distinct dates back, plus the current one, across all 412 stores on each of those dates.

THE TAKEAWAY

Note there is no PARTITION BY here. The frame covers every store's row for each of those four dates, which is the shape GROUPS is good at.

The three units, on tied data

UNIT	WHAT '2 PRECEDING' MEANS
ROWS	Two rows back. Ties are ordinary rows and are counted individually.
RANGE	Ordering value minus two. Arithmetic, so the column must be numeric or a date.
GROUPS	Two distinct ordering values back, whatever number of rows those hold.
All three	Identical when the ordering column is unique. Different the moment it is not.

THE TAKEAWAY

The last row is the whole module. Three units that agree on clean data and disagree on real data, chosen by a word most people never type.

Looking forward

```
1 SELECT sale_date, daily_sales,  
2        SUM(daily_sales) OVER (  
3          PARTITION BY store_key  
4          ORDER BY sale_date  
5          ROWS BETWEEN 1 FOLLOWING AND 7 FOLLOWING  
6        ) AS next_week_sales  
7 FROM agg_store_day;
```

- 5 The next seven rows, excluding today. The last seven rows of each store return partial sums, and the final row returns null.

THE TAKEAWAY

Frames can start in the future. That is legitimate for backtesting and analysis, and it is never legitimate in a measure that will be quoted as of today.

EXCLUDE, briefly

- ◆ **EXCLUDE CURRENT ROW** removes only this row from the frame. A leave-one-out average, in one clause.
- ◆ **EXCLUDE TIES** removes the peers but keeps this row. **EXCLUDE GROUP** removes this row and its peers.
- ◆ **Support is thin.** Postgres, Oracle, DuckDB, SQLite and Trino have it. Redshift, Snowflake, BigQuery and SQL Server do not.
- ◆ **Without it, subtract.** Sum over the frame minus this row's value gives the same answer everywhere.

THE TAKEAWAY

Comparing a row against its group without including itself is a real need. On most engines it is arithmetic rather than syntax.

Leave-one-out, without EXCLUDE

```
1 SELECT store_key, net_amount,  
2         (SUM(net_amount) OVER w - net_amount)  
3         / NULLIF(COUNT(*) OVER w - 1, 0) AS peer_avg  
4 FROM fact_sales_line  
5 WINDOW w AS (PARTITION BY region_key);
```

3 Subtract this row from both the numerator and the denominator. The NULLIF guards a partition of one.

THE TAKEAWAY

How this store compares with its region excluding itself. Without the exclusion, a large store is partly being compared with itself and looks closer to average than it is.

Frames and NULL

- ◆ **Nulls are in the frame like any other row.** The aggregate then skips them, which is module two.
- ◆ **So a moving average over a series with nulls has a shrinking denominator,** silently, exactly as AVG always does.
- ◆ **And under RANGE, nulls are peers of each other,** so they form one enormous group at whichever end they sort to.
- ◆ **COUNT over the same frame exposes it.** One extra window, and the denominator stops being invisible.

THE TAKEAWAY

Nothing here is new. It is module two's rules operating inside a frame, which is why they are worth having learned separately.

What frames cost

- ◆ **UNBOUNDED PRECEDING to CURRENT ROW is cheap.** One accumulator, carried forward, one pass after the sort.
- ◆ **A sliding frame is not.** Rows leave the window as well as entering it, so the engine cannot simply accumulate.
- ◆ **Wider frames cost more,** though good engines keep a running window rather than re-summing from scratch.
- ◆ **RANGE costs more than ROWS,** because peer boundaries have to be found before the frame can be drawn.

THE TAKEAWAY

The cheapest frame is the one people write by accident. The expensive ones are the moving averages, and their cost scales with the width you chose.

Frames you did not write

- ◆ **BI tools generate windows.** A running total toggle in a dashboard emits SQL, and you rarely see which frame it chose.
- ◆ **So does generated and suggested SQL.** The default frame is exactly the kind of thing that gets omitted as unnecessary.
- ◆ **Read the generated query once,** for any measure that matters. Most tools will show it, and most people never look.
- ◆ **If the tool cannot express a frame,** compute the measure in the warehouse and let the tool display a column.

THE TAKEAWAY

The defect in this module does not require anybody to have written SQL. It requires only that somebody ticked a box labelled running total.

A dashboard computes a ninety-day trailing average of sales per store directly over `fact_sales_line`, using `ROWS BETWEEN 89 PRECEDING AND CURRENT ROW` after ordering by timestamp within store. The fact table holds 2.41 billion rows.

The query takes eleven minutes and spills heavily. The dashboard is refreshed hourly and the team wants it under thirty seconds. The measure itself is not in dispute.

What is the right change?

- A Narrow the frame to thirty days, which reduces the sliding window the engine has to maintain.
- B Increase the cluster size, since the query spills and is therefore memory-constrained.
- C Aggregate to one row per store per day first, then apply the frame to roughly half a million rows.
- D Replace the sliding frame with `UNBOUNDED PRECEDING`, which the engine can compute with a single accumulator.

◆ PAUSE AND DECIDE

What is the right change?

A Narrow the frame

Changes the measure, which the scenario rules out, and still sorts 2.41 billion rows. The frame width is not where the cost is.

C Aggregate to daily first

412 stores over about 1,300 days is half a million rows. The same frame, the same answer, on four thousand times less data.

B Bigger cluster

Reduces the spilling and leaves the sort in place. It buys time against a table that grows by roughly two million rows a day.

D Use UNBOUNDED PRECEDING

Genuinely cheaper, and it computes a lifetime cumulative average rather than a ninety-day trailing one. A different measure.

THE SIGNAL

"Ninety days" is a daily measure being computed at line grain. The frame is not the problem. Two point four billion rows are, and 2.41 billion of them did not need to be there.

Choosing a frame, in four questions

01

Needed

Is the function frame-sensitive? Ranking functions take none, so stop here.

02

Unit

Counting rows, values or value groups? That picks ROWS, RANGE or GROUPS.

03

Ends

Write both, with BETWEEN, even when one of them is the default.

04

Ties

Ask what should happen when two rows are equal. The unit has already answered.

THE TAKEAWAY

Step four is the one to do out loud. Under ROWS ties are separate, under RANGE they are one, and under GROUPS they are one unit of distance.

Testing a frame

- ◆ **Build a tiny fixture with deliberate ties.** Six rows, three sharing a value, is enough to separate ROWS from RANGE.
- ◆ **Assert the last row equals the partition total** for any running measure. That one holds under both, so it proves nothing on its own.
- ◆ **Assert a middle row too.** That is the assertion that distinguishes them, and it is the one nobody writes.
- ◆ **Real data will not do this for you**, because the ties may not exist yet and the test would pass today.

THE TAKEAWAY

A test built from production data tests today's data. A fixture with deliberate ties tests the query, which is the thing that will still be there next year.

The fixture that separates them

```
1 -- six rows, three tied on d
2 -- d=1 v=10    d=2 v=20    d=2 v=30    d=2 v=40    d=3 v=50    d=4 v=60
3
4 -- ROWS  running: 10, 30, 60, 100, 150, 210
5 -- RANGE running: 10, 100, 100, 100, 150, 210
6
7 -- assert running_total = 100 at the second row
8 -- passes under RANGE, fails under ROWS
```

5 Rows one, five and six agree in both columns. Only the three tied rows differ, and only two of those three.

THE TAKEAWAY

Eight lines of fixture, one assertion on a middle row, and the difference between the two units is permanently visible in your test suite.

The confusions, settled

SYMPTOM	THE CAUSE
LAST_VALUE returns the current row	Default frame ends at CURRENT ROW.
A running total plateaus, then jumps	RANGE peers sharing one cumulative value.
The closing figure is right, the middle is not	The same thing. The last peer is always correct.
A moving average spans too many days	ROWS counted rows across a gap in the series.
A six-row window contains seven rows	No BETWEEN, so the frame runs through to CURRENT ROW.
The query errors on Redshift and not Postgres	Redshift requires an explicit frame. It is doing you a favour.

THE TAKEAWAY

Five of these produce a plausible number. The sixth produces an error, and it is the only one anybody finds on the day they write it.

Five habits for frames

- ◆ **Write the frame on every aggregate window.** Every one, even when the engine would have supplied it.
- ◆ **Write BETWEEN and both ends.** The shorthand hides an off-by-one that nobody reviews.
- ◆ **Default to ROWS** unless you specifically want peers or an interval, in which case say so and say why.
- ◆ **Carry a COUNT over the same frame** whenever the measure is an average. The denominator stops being invisible.
- ◆ **Treat a removed frame clause as a change of meaning**, in review, in every direction, on every engine.

THE TAKEAWAY

The first habit alone would have prevented every defect in this module except the moving average. It costs one line and it is not negotiable.

Writing the decision down

- ◆ **A frame is a business decision in syntax.** Seven days or seven rows, peers together or apart, future included or not.
- ◆ **Comment the non-obvious ones.** A one-line note on why RANGE was chosen saves the next person reversing it.
- ◆ **An uncommented RANGE looks like a typo for ROWS,** and somebody will eventually 'correct' it.
- ◆ **The same is true in the other direction,** which is how scenario one happened in the first place.

THE TAKEAWAY

Every defect in this module was somebody removing or omitting six words. A comment is the only thing that makes those six words look deliberate.

If a window has an ORDER BY and no frame,
somebody chose a frame. It was not you.

You now have all three questions. **Module six** is the family that lives on the edges of the frame.